IDS Resiliency Against Evasion Attacks

Teaundra R. Thompson

Charleston Southern University

In Fulfillment of ACE-Cyber R&D Program

Professor Littleton

January 16th, 2026

This paper details the process and findings of conducting my ACE-Cyber R&D research project. The chosen topic for the project was “AI Intrusion Detection System’s Resilience to Adversarial Evasion – PoC.” I developed a ML-based IDS to determine the resilience of an IDS when faced with adversarial evasion attacks. I utilized free and open-source software and datasets to construct a baseline IDS with an ML component. The findings confirmed the initial hypothesis, highlighting that while the model detects high volume attacks with 100% accuracy, the performance significantly reduced when faced with low-volume evasion examples, with detection accuracy dropping as low as 24%. This project was completed over the course of three months.

ML, MLP, DoW, Zeek, Adversarial Evasion, IDS, NIDS, pcap.

IDS Resiliency Against Evasion Attacks

The topic chosen to complete this project was "AI Intrusion Detection System (IDS) Resilience to Adversarial Evasion – PoC." The objective of my research was to investigate the resilience of an AI-based IDS when subjected to adversarial evasion attacks and demonstrate the risks of AI in cyber defense and inform red-teaming practices for Department of War (DoW) AI models. The project involved training a baseline ML IDS model on an open-source network traffic dataset, then crafting and applying adversarial examples to measure the model's detection accuracy drop. The PoC (Proof-of-Concept) aimed to demonstrate the risks of AI in cyber defense and inform red-teaming practices for DoW AI models. To create the ML IDS, I had to acquire two components, baseline IDS software to generate logs and a ML model to interpret the logs. For the baseline IDS software, I used Zeek version 8.0.1, and for the ML model, I used a Multi-Layer Perceptron (MLP) model with five layers. For crafting adversarial evasion attacks, I focused on typical evasion attacks instead of poisoning attacks for the sake of time. Poisoning attacks are done over an extended length of time and are relative to online learning systems, which was not suited for my research since there was a three-month constraint (Chio & Freeman, 2018). Crafting evasion examples are considerably quicker as they give instantaneous results instead of having to repeatedly inject arbitrary data into the model and wait for the model parameters to shift.

Literary Review

Before constructing and implementing the PoC, I needed to conduct a literature review to determine how I would proceed with my research. I needed to know what the current trends in adversarial evasion attacks against ML-driven IDSs were, and what type of datasets and ML models were being used to observe these trends. One source that helped me determine what types of adversarial attacks I wanted to generate was *Machine Learning and Security: Protecting Systems with Data and Algorithms* by Chio and Freeman. Since my model was not going to be an online learning system, attempting to "poison" the model would be futile since it is only trained once and no input after training

affect its prediction parameters (Chio & Freeman, 2018)[1]. Evasion attacks can affect any ML model because it does not target the "training phase." This made adversarial evasion more applicable to my project since I did not need an online learning system. During my literature review I also had to determine what type of IDS I wanted to create. There are two main types: a Network Intrusion Detection System (NIDS), and a Host Intrusion Detection System (HIDS). I was already predisposed towards choosing a NIDS, but I needed confirmation to go with the selection. From a journal article titled "Anomaly network intrusion detection system based on NetFlow using machine/deep learning" I discovered that NIDSs are more scalable than HIDSs and are not confined by compatibility issues that may arise when dealing with different operating systems (Adli et al., 2023). After reading this article I determined that a NIDS would be more general and would make my project easily reproducible without the confines of the architecture of different operating systems. I also determined what type of ML model I wanted to create during my literature review. The journal article "Model Evasion Attack on Intrusion Detection Systems using Adversarial Machine Learning" was a hypthetico-deductive research paper on how model evasion attacks can effectively reduce the detection accuracy of a MLP-based NIDS (Ayub et al., 2020). The MLP model intrigued me because in the research article it received a baseline detection accuracy of over 99% which showed that it did well with detecting typical attacks which was important for my project. Determining the type of adversarial attacks, IDS, and ML model I wanted to implement were my biggest findings during my literature review.

Implementation

Once I decided on the type of IDS, the type of adversarial attacks, and the ML model I wanted to create, the testbed setup and implementation was ready to begin. I constructed a virtual machine (VM) environment for the testbed. I used the hypervisor VirtualBox and created an Ubuntu VM to host the

[1] Online learning systems dynamically retrain ML models with user inputs to automatically adjust model parameters.

baseline IDS software and capture network traffic.[2] Additionally, I had a Kali Linux VM that was previously configured and set up in VirtualBox, so I utilized that VM to generate evasion traffic for a few test cases.[3] The host machine for VirtualBox was a Windows 11 Home laptop with 32 GB of RAM and 1.38 TB of storage. Furthermore, I needed to determine the baseline IDS software I was going to utilize to generate logs in the VM for the ML model to analyze. I initially intended to utilize Security Onion, as I found the platform to be robust and it had a graphical user interface (GUI) which would make interpreting logs simple. After installing Security Onion and attempting to set it up I encountered a problem; it was too resource intensive for the VM, and it was essentially an enterprise grade platform which was unnecessary for a baseline IDS for log generation. Therefore, I chose to install and setup a new software, Zeek. Zeek was lightweight and it generated detailed logs necessary for ML analysis. For simplicity, I chose to use only one of Zeek's logs, the connection log (conn.log), for analysis because it provided a basic summary of all network traffic. Moreover, I disabled the live-monitoring feature in Zeek to ensure no personally identifiable information (PII) was captured during testing. After having the baseline IDS setup and configured, I needed to create the MLP model. I utilized a cloud environment, Google Colab, to train and host the model for simplicity. Since I am a college student, I was able to obtain a Google Colab Pro subscription for free, which allotted me 100 compute credits monthly with rollover.[4] Using Google Colab for the MLP training and hosting put less strain on the VM environment and was easy to implement and utilize.

Dataset and MLP Model

To create and train the MLP model I had to determine what open-source dataset I wanted to utilize. I initially chose the NSL-KDD dataset because it was small with around 25,000 samples. The

[2] Ubuntu VM was version 24.04.

[3] The Kali Linux VM was setup months prior for a home lab experiment, no pertinent data was present in the VM. The Kali Linux VM was version 2025.2.

[4] Only 8.28 compute units during the course of the project.

problem with that dataset was the features were not compatible with the Zeek conn.log. I could not map the Zeek conn.log features to the features in the NSL-KDD dataset without causing the MLP to drastically misclassify typical anomalous traffic as benign. Therefore, I had to change the dataset to one that was compatible with Zeek's connection log format and alter the dataset preprocessing code. Thankfully I discovered the UFW-ZeekData24 dataset which was in Zeek data format and featured eight attack classes classified according to the MITRE ATT&CK framework (Elam et al., 2025). For testing purposes, I excluded two of the attack classes, Exfiltration and Defense Evasion. I excluded Exfiltration because I was not intending on creating any adversarial examples portraying such activity. I purposely excluded Defenses Evasion to be able to realistically test the model against evasion techniques adversaries would use.[5] With those two classes excluded, I was left with the Benign, Credential Access, Initial Access, Persistence, Privilege Escalation, and Reconnaissance classes. The portion of the dataset that I used had a total of 95,522 samples. Despite having a decent sized dataset, there was a major class imbalance. The Benign class had 47,906 samples, Credential Access had 43,494 samples, Initial Access had 561, Persistence had 326, Privilege Escalation had 326, and Reconnaissance had 2,909. I handled some of the imbalance by calculating the class weights. By calculating the class weights I made the model pay more attention to the minority classes by assigning a higher value to them letting the model know that misclassifying the minority classes is riskier than misclassifying the majority classes. For the actual MLP model I went with five layers; an input/hidden layer, second hidden layer, third hidden layer, fourth hidden layer, and an output layer.[6] Upon testing the model it achieved an accuracy of 95.95% with a weighted F1-Score of 97.14% and a macro F1-Score of 56.22%. Considering the class imbalance, these were satisfactory results from the testing phase, and the model was ready to undergo the experimentation phase.

---

[5] Evasion techniques are typically not present in training data, making them effective for adversaries.

[6] Five layers were used because it was the model architecture for the MLP model when it was trained on the NSL-KDD dataset.

Test Cases

There were seven test cases conducted. Test case one (TC1) consisted of establishing a baseline accuracy of the model. I generated a pcap with non-anomalous traffic then fed the pcap into Zeek to generate the conn.log. Subsequently, I uploaded the conn.log to the Google Colab environment and had the MLP model analyze the data. The model gave an output of "No Anomalies Detected" since the amount of connections flagged as anomalous did not exceed the 40% threshold I set for the model. Despite the output, there were still some false positives. For TC1 the false positive rate (FPR) was 31.7%. For TC2, I generated a pcap that displayed a DDoS attack with over 1,000,000 connections. Despite the model not being trained on the Impact MITRE ATT&CK tactic, it detected the anomalous traffic with a true positive rate (TPR) of 100% showcasing the models ability to detect high volume attacks. TC3 was an adversarial evasion example. I generated a pcap using a Python/Scapy script generated by Claude AI to simulate overlapping fragmentation. I also generated benign traffic in the same pcap to hide the evasive techniques; the conn.log had a total of 480 malicious connections. The model detected the attack partially with a TPR of 66.67% and a FPR of 16.03%. TC4 was also an adversarial evasion example. The pcap for this test case was sourced from malware that was analyzed on Recorded Future Triage.[7] The malware was Spyware that made discrete TCP connections to a C2 (Command and Control) server, with a total of 46 malicious connections. The model detected the attack partially with a TPR of 23.91%. TC5 was also an evasion example; I generated a pcap that showcased intermittent password cracking attempts mixed in with benign traffic. The model interpreted the password cracking attempts as Reconnaissance instead of Credential Access which led to a TPR of 100% since Reconnaissance had a higher-class weight than Credential Access. TC6 was to test the model's prediction of actual Reconnaissance Tactics; I simulated port scanning traffic, and the model accurately detected all of the connections with a TPR of 100%. TC7 was my final evasive test case; I simulated intermittent port

[7] Link to where the pcap was sourced on RF Triage: https://tria.ge/251122-d7a5macm6s/behavioral1.

scanning with benign traffic mixed into the pcap capture. The model detection of this test case was a TPR of 100% with a FPR of 31.48%.

Results

Below is Table 1 with the results from all the executed test cases. It contains the TPR, FPR, MITRE Tactic, and traffic volume displayed from each test case.

Table 1

Model's Resiliency to High and Low Volume Attacks

Test Case	Description	TPR	FPR	Volume	MITRE Tactic
TC1	Normal Traffic (Baseline)	N/A	31.7%	N/A	Benign
TC2	DDoS	100%	N/A	High	Impact
TC3	Overlapping Fragmentation	66.67%	16.03%	Low	Defense Evasion
TC4	Spyware	23.91%	N/A	Low	Defense Evasion
TC5	Evasive Password Cracking	100%	24.21%	Low	Defense Evasion + Credential Access
TC6	Port Scanning	100%	N/A	High	Reconnaissance
TC7	Port Scanning (Evasive)	100%	31.48%	Low	Defense Evasion + Reconnaissance

Analysis of Results

Overall, the model performed well on high volume attacks such as DDoS (TC2) but struggled significantly against low volume evasion attacks such as spyware (TC4). This showcases the effectiveness of adversarial evasion attacks and highlights the need for training and model configuration to mitigate the effectiveness of such attacks. While not a primary factor, something to note is that the individual connections flagged as anomalous were attributed a confidence score based on the model's confidence of its prediction. These scores help to understand the model's misclassifications. TC1 displayed an

unusually high FPR for a baseline test. Out of the total 2,952 connections, 26.6% of them were flagged as Reconnaissance with majority of them having a high confidence score of 95%. The remaining 5.1% of misclassifications were distributed across the remaining classes with low confidence scores. All of the connections that were flagged as Reconnaissance were connection attempts made to port 443 or to port 80. The large amount of false Reconnaissance flags stems from the training data for the Reconnaissance tactic; 53% of the samples in this set were connections made to port 443 and 80. Additionally, the calculation of class weights to handle the data imbalance also contributed to the false positives. Thus, due to the nature of the training data, it resulted in TC1 having a FPR of 31.7%. Likewise, TC7 also had a high FPR due to the model's interpretation of normal traffic. The traffic in TC7 was both normal and anomalous, simulating evasive behavior. Despite the attempt at evasion, all of the port scanning traffic was detected accurately with a TPR of 100%. Nevertheless, the model flagged 14.5% of normal traffic as Reconnaissance with a majority high confidence score above 90%, and 12.3% of normal traffic as Initial Access with low confidence, and the remaining 4.7% being distributed amongst the remaining classes with low confidence. The connections flagged as Initial Access are not of concern as all the flagged connections have low confidence scores below 30%. TC2 was the largest volume attack, and the model achieved a TPR of 100% despite not having any training data on the Impact MITRE tactic. As a result, the model classified all of the connections as Persistence with confidence scores less than 40%. TC4 lacks a FPR as a result of all the connections being anomalous, thus nullifying the chance of false positives. TC5 displays a TPR of 100% with a FPR of 24.21%; the reason for these results is due to the misclassification of the anomalous connections. This test case was supposed to test the model's ability to classify the Credential Access MITRE tactic, but all of the Credential access connections were flagged as Reconnaissance with high confidence. The reason for that result also stems from the training data; the samples for the Credential Access tactic only display connections to port 4848, and during the execution of the test case I was only making connections to port 22. Thus, resulting in the

misclassification of the intended MITRE Tactic. This highlights the limitations of ML when it comes to classifying traffic, because the training data may not accurately depict different variations of traffic within a class leading to misclassifications.

Conclusion

This research project illuminated the vulnerabilities of adversarial evasion attacks against AI or ML based Intrusion Detection Systems. As seen in Table 1, the model had no difficulties detecting high volume attacks but had difficulty accurately detecting low-volume evasive attacks. This result shows how effective adversarial evasion can be when executed by an experienced bad actor and the limitations of AI/ML of detecting and classifying such attacks. This correlates to my hypothesis that the model would have a higher TPR against adversarial examples that have a higher frequency (large volume attacks), than examples that have a lower frequency (small volume attacks). The results directly support my hypothesis by proving the model performs less considerably on low-volume attacks compared to high-volume attacks. It is important to note the vulnerabilities of AI-based IDS as such systems are critical in cybersecurity infrastructure, and even more so to the DoW. Knowing the vulnerabilities of these systems before adversaries is vital because if not then adversaries can exploit the vulnerabilities undetected and cause costly and excessive remediation. It is important to determine how these systems behave when faced with adversarial attacks, so the vulnerabilities can be mitigated.

Future Work

Future work should focus on retraining the model with a more balanced dataset to prevent the risk of misclassifications and help the model distinguish between normal traffic and low-volume attacks. Such a dataset can be acquired by combining existing compatible datasets or manually modifying existing datasets to include desired attack types. For instance, the UFW-ZeekData24 dataset can be combined with earlier releases of UFW-ZeekData to create a balanced sample set, or synthetic samples can be manually created and added to the dataset. Further research should be done on the

performance of various ML models when faced with adversarial evasion to determine the impact of model type on resiliency. Also, utilizing multiple models, also known as ensemble learning, would greatly improve accuracy as the different models would compensate for their counterparts weaknesses (Ganesan & Sarac, 2021). An example of ensemble learning would be a combination of a decision tree model and a neural network. Decision trees are able to detect broad patterns, while neural networks are effective at detecting subtle patterns. While no system can be completely invulnerable, conducting this future work will greatly improve the security and robustness of AI/ML based IDSs.

References

Adli, T. B., Amokrane, S.-B. B., Pavlović, B. Z., Laidouni, M. Z. M., & Benyahia, T. A. A. (2023). Anomaly network intrusion detection system based on NetFlow using machine/deep learning. *Military Technical Courier / Vojnotehnicki Glasnik*, *71*(4), 941–969. https://doi.org/10.5937/vojtehg71-46058

Ayub, Md. A., Johnson, W. A., Talbert, D. A., & Siraj, A. (2020). Model Evasion Attack on Intrusion Detection Systems using Adversarial Machine Learning. *2020 54th Annual Conference on Information Sciences and Systems (CISS)*, 1–6. https://doi.org/10.1109/CISS48834.2020.1570617116

Chio, C., & Freeman, D. (2018). *Machine Learning and Security: Protecting Systems with Data and Algorithms*. O'Reilly Media, Inc.

Elam, M., Mink, D., Bagui, S. S., Plenkers, R., & Bagui, S. C. (2025). Introducing UWF-ZeekData24: An Enterprise MITRE ATT&CK Labeled Network Attack Traffic Dataset for Machine Learning/AI. *Data*, *10*(5), 59. https://doi.org/10.3390/data10050059

Ganesan, A., & Sarac, K. (2021). Mitigating Evasion Attacks on Machine Learning based NIDS Systems in SDN. *2021 IEEE 7th International Conference on Network Softwarization (NetSoft)*, 268–272. https://doi.org/10.1109/NetSoft51509.2021.9492526